

FDU 并行计算 1. 并行计算机

本文参考以下教材:

- 并行计算: 结构·算法·编程 (第 3 版, 陈国良) 第 1, 2, 3, 4 章

欢迎批评指正!

1.1 Introduction

并行计算是指在并行计算机或分布式计算机等高性能计算系统上进行的计算过程.

在不同的并行计算机模型上, 并行算法的设计方法是不同的.

1.1.1 结构模型

并行计算机大致可分为:

- **单指令多数据流计算机** (Single-Instruction Multiple-Data, **SIMD**):
由一个控制单元同时向多个处理单元发出同一条指令, 使其并行地对不同数据执行相同运算.
- **多指令多数据流计算机** (Multiple-Instruction Multiple-Data, **MIMD**):
由多个处理单元各自独立执行不同指令、处理不同数据, 适合通用并行计算.
 - **PVP** (Parallel Vector Processor, **并行向量处理机**)
以向量指令为核心, 通过对长向量数据进行流水线并行运算来提高吞吐率.
 - **SMP** (Symmetric Multiprocessor, **对称多处理机**)
多个功能对等的处理器共享统一的物理内存和操作系统, 实现紧耦合并行.
 - **MPP** (Massively Parallel Processor, **大规模并行处理机**)
由大量处理节点组成, 各节点通常具有私有内存, 通过高速互连网络进行通信.
 - **COW** (Cluster of Workstations, **工作站机群**)
通过局域网将多台独立工作站连接起来, 利用消息传递实现松耦合并行计算.
 - **DSM** (Distributed Shared Memory, **分布共享存储多处理机**)
在物理上分布的内存之上提供统一的共享地址空间抽象, 结合共享内存编程模型与分布式实现.

五种 MIMD 的结构模型如下所示.

- B 是存储器总线 MB 和 I/O 总线 IOB 间的接口.
- DIR 是高速缓存目录, LM 是本地存储器, SM 是共享存储器, LD 是本地磁盘.
- P/C 是处理器和高速缓存, VP 是向量处理器.
- NIC 是网络接口电路.

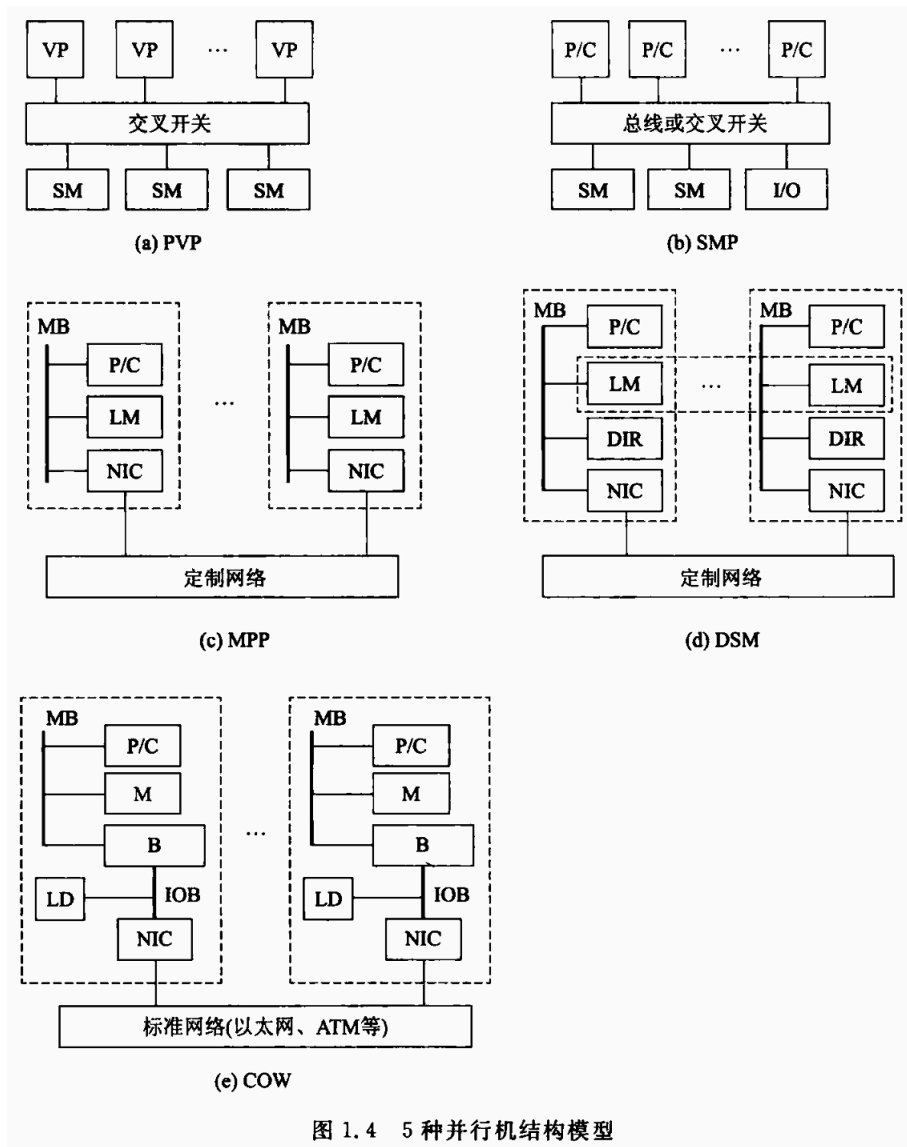


图 1.4 5 种并行机结构模型

属性	PVP	SMP	MPP	DSM	COW
结构类型	MIMD	MIMD	MIMD	MIMD	MIMD
处理器类型	专用定制	商用	商用	商用	商用
互连网络	定制交叉开关	总线、交叉开关	定制网络	定制网络	商用网络 (以太,ATM)
通信机制	共享变量	共享变量	消息传递	共享变量	消息传递
地址空间	单地址空间	单地址空间	多地址空间	单地址空间	多地址空间
属性	PVP	SMP	MPP	DSM	COW
系统存储器	集中共享	集中共享	分布非共享	分布共享	分布非共享
访存模型	UMA	UMA	NORMA	NUMA	NORMA
代表机器	Cray C-90, Cray T-90, 银河 1 号	IBM R50, SGI Power Challenge, 曙光 1 号	Intel Paragon, IBM Option White, 曙光-1000/2000	Stanford DASH, Cray T3D	Berkeley NOW, Alpha Farm

表 1.1 5 种结构特性一览表

1.1.2 访存模型

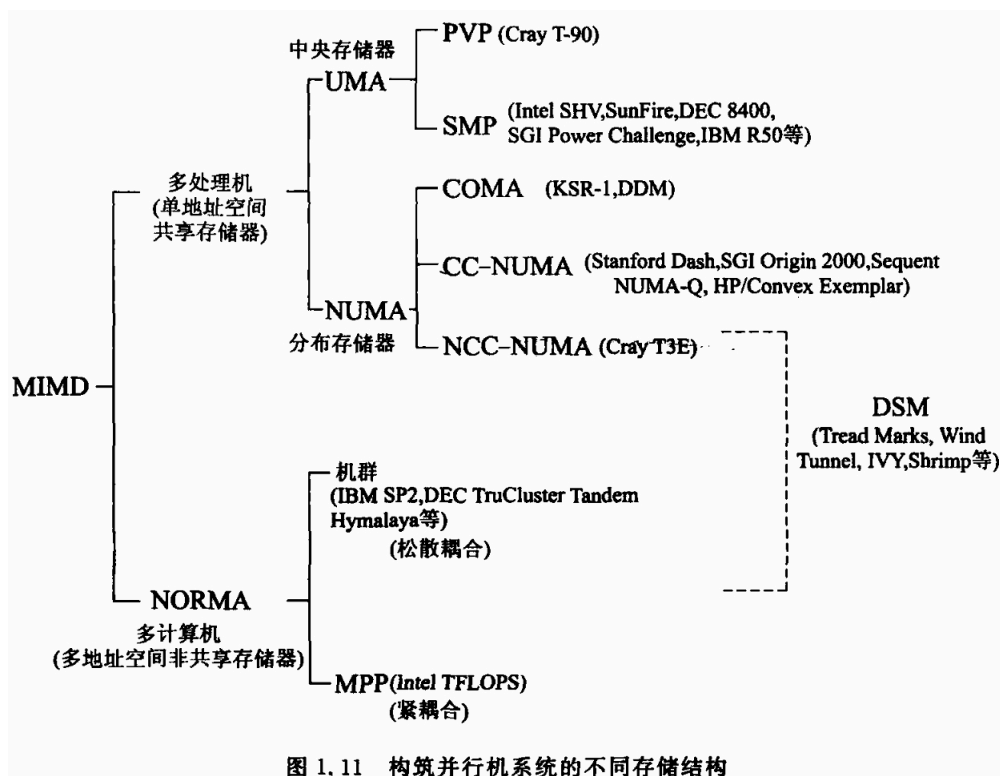
并行计算机的结构模型从硬件层面规定了：

- 处理器之间如何连接
- 内存是集中还是分布
- 处理器访问内存的物理路径.

这些因素直接决定了系统能提供的**访存模型**：

结构模型	访存模型	解释
SIMD	UMA	单控制单元驱动多个处理单元访问共享存储，所有访存延迟在结构上均匀
PVP	UMA	向量处理器通过共享的统一地址空间，对连续或规则数据进行向量化存取
SMP	UMA → CC-NUMA	小规模时均匀访存，规模扩展后通过缓存一致性机制形成非均匀访存
MPP	NORMA	各处理节点仅能访问本地私有内存，节点间通过消息传递进行通信
COW	NORMA	多台独立工作站各自拥有私有内存，通过网络和消息传递实现并行计算
DSM	CC-NUMA / COMA	通过缓存机制维护数据共享，在物理分布存储之上提供共享地址空间抽象

- **UMA (Uniform Memory Access, 均匀存储访问)**
所有处理器共享统一地址空间，访问任一内存位置的延迟在结构上基本相同.
- **NUMA (Nonuniform Memory Access, 非均匀存储访问)**
处理器共享全局地址空间，但访问本地内存与远程内存的延迟不同.
- **CC-NUMA (Coherent-Cache NUMA, 高速缓存一致性非均匀存储访问)**
在 NUMA 结构上通过硬件缓存一致性协议维持共享数据的全局一致性.
- **COMA (Cache-Only Memory Access, 全高速缓存存储访问)**
所有主存以缓存形式组织，数据可在处理器间动态迁移以接近使用者.
- **NORMA (No-Remote Memory Access, 非远程存储访问)**
各处理器仅能访问本地私有内存，不能直接访问远程内存，只能通过消息传递通信.



1.1.3 主要问题

并行计算的主要问题在于存储受限、编程困难、能耗过高以及软硬件生态失衡。

- **存储墙 (Memory Wall)**
处理器计算能力增长远快于存储系统带宽与延迟改进，导致处理器频繁因数据供给不足而空闲。
- **编程墙 (Programming Wall)**
超大规模、异构并行系统结构复杂，缺乏统一的编程模型与工具，使高效并行软件开发极其困难。
- **功耗墙 (Power Wall)**
大规模并行系统为追求性能而导致整体功耗和散热需求急剧上升，限制了系统规模和可持续发展。
- **不平衡的计算机科学生态系统 (Ecosystem Imbalance)**
硬件性能增长过快而软件、算法与应用支撑不足，造成体系结构、软件和应用发展不同步。

1.2 互连网络

互连网络 (Interconnection Networks) 是并行计算机系统中各处理器与存储模块之间传输数据的机制。

1.2.1 系统互连

并行与分布式系统通过分层互连网络连接处理器、存储和 I/O 设备。

节点内采用**处理器总线**、**局部总线**和**存储器总线**实现高速连接；

节点间通过**系统域网络** (SAN, 3-25 m) 形成单一系统，

并可进一步通过**局域网** (LAN, 500 m-2 km) 构成工作站机群，

而**城域网** (MAN, >25 km) 和**广域网** (WAN, 全球) 覆盖范围更大但不作为并行机内部互连重点。

网络性能主要由**局部连接性**、**最坏通信距离**、**全局割裂能力**、**整体吞吐率**和**结构均匀性**共同决定。

- **节点度** (node degree)
一个节点直接相连的边数，反映节点的局部连接能力。
- **网络直径** (network diameter)
网络中任意两个节点之间最短路径长度的最大值，衡量最坏情况下的通信延迟。
- **对剖宽度** (bisection width)
将网络划分为规模相等两部分时所需切断的最少连线数，反映网络的整体连通能力。
- **对剖带宽** (bisection bandwidth)
在最小对剖平面上所有连线可同时传输的最大数据速率，衡量网络的全局通信吞吐能力。
- **对称性** (symmetry)
从任一节点观察网络拓扑均等价，表明网络在结构和通信性能上的均匀性。

1.2.2 静态网络

静态网络 (static network) 是在程序执行期间处理单元之间的连接固定不变的互连网络，而**动态网络** (dynamic network) 通过开关单元按需重构连接以适应不同通信模式。常见的静态网络包括线性阵列、网孔、树、超立方体等。

- **一维线性阵列** (1-D linear array)
节点按直线排列，每个节点仅与相邻节点相连的简单拓扑。
- **二维网孔** (2-D mesh)
节点按二维网格排列，每个节点与上下左右相邻节点相连。
- **树连接网络** (tree network)
节点按二叉树结构连接。
除了根节点和叶节点之外，每个节点只与其父节点和两个子节点相连，即三近邻连接。
- **超立方网络** (hypercube)
节点按二进制地址组织，两个节点若地址仅一位不同则直接相连。
- **立方环** (cube-connected cycles, CCC)
将超立方体中每个节点替换为一个环，以降低节点度而保持较小直径。

表 2.1 静态互连网络特性一览表						
网络名称	网络规模	节点度	网络直径	对剖宽度	对称	链路数
线性阵列	N 个节点	2	$N-1$	1	非	$N-1$
环形	N 个节点	2	$\lfloor N/2 \rfloor$ (双向)	2	是	N
2-D 网孔	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$2(\sqrt{N}-1)$	$\sqrt{N}$	非	$2(N-\sqrt{N})$
Illiac 网孔	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$\sqrt{N}-1$	$2\sqrt{N}$	非	$2N$
2-D 环绕	$(\sqrt{N} \times \sqrt{N})$ 个节点	4	$2\lfloor \sqrt{N}/2 \rfloor$	$2\sqrt{N}$	是	$2N$
二叉树	N 个节点	3	$2(\lceil \log N \rceil - 1)$	1	非	$N-1$
星形	N 个节点	$N-1$	2	$\lfloor N/2 \rfloor$	非	$N-1$
超立方	$N=2^n$ 个节点	n	n	$N/2$	是	$nN/2$
立方环	$N=k2^k$ 个节点	3	$2k-1+\lfloor k/2 \rfloor$	$N/(2k)$	是	$3N/2$

注：表中对数以 2 为底。

1.2.3 动态网络

动态网络的典型形式包括总线网络、交叉开关网络和多级互连网络.

- **总线系统** (bus system)
多个节点共享一条通信通道，通过仲裁机制分时访问.
结构简单、成本低，但并发能力差、易发生冲突.
- **交叉开关系统** (crossbar switch system)
通过交换矩阵为任意输入与输出节点提供独立的并行连接，
可提供最高带宽和最优选路性能，但硬件成本极高.
- **多级互连网络** (Multistage Interconnection Network, MIN)
由多个小规模交换单元按级联方式构成，在成本与性能之间折中，
具有良好扩展性，但通信延迟随规模呈对数增长.

表 2.2 动态互连网络特性一览表			
网络特性	总线系统 n 台处理器, 总线 宽度为 ω 位	多级互连网络 $n \times n$ MIN 采用 $k \times k$ 开 关, 线宽为 ω 位	交叉开关 $n \times n$ 交叉开关 线宽为 ω 位
最小时延	恒定(轻负载)	$O(\log_k n)$	恒定
每台处理器带宽	$O(\omega/n) \sim O(\omega)$	$O(\omega) \sim O(n\omega)$	$O(\omega) \sim O(n\omega)$
开关复杂性	$O(n)$	$O(n \log_k n)$	$O(n^2)$
连线复杂性	$O(\omega)$	$O(n\omega \log_k n)$	$O(n^2 \omega)$
连接特性	一次只能一对一	是阻塞型网络	全置换

1.3 加速比定律

1.3.1 性能指标

下表列出了并行计算机的一些基本性能指标.

表 4.1 并行机基本性能参数一览表

名称	符号	含意	单位
机器规模	n	处理器的数目	无量纲
时钟速率	f	时钟周期长度的倒数	MHz
工作负载	W	计算操作的数目	MFLOP
顺序执行时间	T_1	程序在单处理机上的运行时间	s(秒)
并行执行时间	T_n	程序在并行机上的运行时间	s(秒)
速度	$R_n = W/T_n$	每秒百万次浮点运算	MFLOPS
加速	$S_n = T_1/T_n$	衡量并行机有多快	无量纲
效率	$E_n = S_n/n$	衡量处理器的利用率	无量纲
峰值速度	$R_{\text{peak}} = nR'_{\text{peak}}$	所有处理器峰值速度之积, R'_{peak} 为一个处理器的峰值速度	MFLOPS
利用率	$U = R_n/R_{\text{peak}}$	可达速度与峰值速度之比	无量纲
通信延迟	t_0	传送 0 字节或单字的时间	μs
渐近带宽	r_{∞}	传送长消息通信速率	MBps

工作负载可通过执行时间、指令条数和浮点运算数来度量。

执行时间是程序从开始到结束所消耗的总时间，包括 CPU 时间、访存时间和 I/O 时间等。

若不考虑重叠，则并程序的执行时间可表示为：

$$T = T_{\text{compute}} + T_{\text{parallel}} + T_{\text{communicate}}$$

- T_{compute} : 计算时间。
- T_{parallel} : 并行开销时间，包括进程管理 (创建、结束、切换)、组操作和进程查询等。
- $T_{\text{communicate}}$: 通信时间，包括同步操作 (例如锁)、通信操作 (例如共享变量读写) 和聚合操作等。

指令条数通常以 MIPS (每秒百万条指令) 为单位来衡量。

浮点运算数通常以 FLOPS (每秒浮点运算次数) 为单位表示，其估算可依据以下经验规则：

- 赋值操作、变址计算等不单独计入，折算为 0 FLOPS。
- 单独赋值、加、减、乘、比较、数据类型转换等基础运算折算为 1 FLOPS。
- 除法与开方运算折算为 4 FLOPS。
- 正弦、余弦、指数等复杂运算折算为 8 FLOPS。

存储器层次结构的每一层都可用三个参数表征：

- **容量** C : 存储器能保存的数据字节数。
- **延迟** L : 读取一个字所需的时间。
- **带宽** B : 单位时间内可传输的数据字节数。

内存带宽指数据从内存传输到处理器的速度，由存储器总线和存储部件决定。

例如，在流水线 RISC 机器中，64 位 (8 字节) 加法指令通常可在单个时钟周期内完成。

若时钟为 100 MHz，则存储带宽 $B \approx 3 \times 8 \text{ B} \times 100 \text{ MHz} = 2.4 \text{ GBps}$ 。

点到点通信时间的线性模型：

$$t(m) = t_0 + \frac{m}{r},$$

其中 m 是消息字节数, t_0 为通信启动时间 (单位为 μs),
 r_∞ 为渐近带宽 (单位为 MBps), 即传送无限长消息时的通信速率.
 此外, 我们还可引入特定性能 π_0 和半峰值长度 $m_{1/2}$,
 分别代表短消息带宽和达到一半渐近带宽所需的消息长度, 满足:

$$t_0 = \frac{m_{1/2}}{r_\infty} = \frac{1}{\pi_0}.$$

测量方法:

- **乒乓法 (Ping-Pong):**
 节点 0 发送 m 个字节给节点 1; 节点 1 接收后立即回送节点 0.
 总时间除以 2 即为单次发送或接收的通信时间.
- **热土豆法 (Hot-Potato) / 救火队法 (Fire Brigade):**
 消息依次从节点 $0 \rightarrow 1 \rightarrow \dots \rightarrow n-1 \rightarrow 0$,
 最后总时间除以 n , 得到平均点到点通信时间.

1.3.2 Amdahl 定律

Amdahl 定律针对**固定问题规模**的并行计算, 分析处理器数量增加对加速比的影响.
 设程序中串行部分比例为 f , 可并行部分的比例为 $1 - f$, 使用 p 个处理器时, 加速比为

$$\begin{aligned} S(p) &= \frac{W_{\text{serial}} + W_{\text{parallel}}}{W_{\text{serial}} + W_{\text{parallel}}/p} \\ &= \frac{p}{1 + (p-1)f}, \end{aligned}$$

其中 $f = W_{\text{serial}}/(W_{\text{serial}} + W_{\text{parallel}})$.
 当处理器数 $p \rightarrow \infty$ 时, 加速比的上限为

$$S_{\max} = \frac{1}{f}.$$

这表明**串行部分决定了并行加速的理论上限**, 即使无限增加处理器, 加速比也无法超过 $1/f$.

考虑到并行运行的额外开销 $W_{\text{extra}}(p)$, 加速比还会进一步降低.

$$\begin{aligned} S(p) &= \frac{W_{\text{serial}} + W_{\text{parallel}}}{W_{\text{serial}} + W_{\text{parallel}}/p + W_{\text{extra}}(p)} \\ &= \frac{p}{1 + (p-1)f + W_{\text{extra}}(p)/(W_{\text{serial}} + W_{\text{parallel}})}. \end{aligned}$$

串行比例和并行带来的额外开销越大, 可获得的加速效果越小.

1.3.3 Gustafson 定律

Gustafson 定律针对**固定执行时间、随处理器数增大而放大问题规模**的并行计算情形，强调在实际应用中增加处理器应相应扩大计算规模以提高精度或分辨率。

设程序在单处理器下，串行部分所占比例为 f ，并行部分为 $1 - f$ ，使用 p 个处理器时，Gustafson 给出的加速比为

$$\begin{aligned} S(p) &= \frac{W_{\text{serial}} + pW_{\text{parallel}}}{W_{\text{serial}} + pW_{\text{parallel}}/p} \\ &= f + p(1 - f) \\ &= p - (p - 1)f, \end{aligned}$$

其中 $f = W_{\text{serial}}/(W_{\text{serial}} + W_{\text{parallel}})$ 。

当 p 足够大时，加速比近似与处理器数**线性增长**，其斜率为 $1 - f$ 。

这表明在放大问题规模的前提下，串行部分不再构成主要瓶颈，结论相较 Amdahl 定律更为乐观。

若考虑并行执行的额外开销 $W_{\text{extra}}(p)$ ，则加速比可修正为

$$\begin{aligned} S(p) &= \frac{W_{\text{serial}} + pW_{\text{parallel}}}{W_{\text{serial}} + pW_{\text{parallel}}/p + W_{\text{extra}}(p)} \\ &= \frac{f + p(1 - f)}{1 + W_{\text{extra}}(p)/(W_{\text{serial}} + W_{\text{parallel}})} \\ &= \frac{p - (p - 1)f}{1 + W_{\text{extra}}(p)/(W_{\text{serial}} + W_{\text{parallel}})}, \end{aligned}$$

要获得接近线性的加速，需使额外开销不随 p 增大而显著增长，但在实际系统中往往较难实现。

1.3.4 Sun-Ni 定律

Sun-Ni 定律针对**存储容量受限但问题规模可扩展**的并行计算情形，对 Amdahl 定律与 Gustafson 定律进行了统一和推广。

设单处理器系统在使用全部存储容量 M 时，完成工作负载 W ，其中串行比例为 f ，并行比例为 $1 - f$ 。

在使用 p 个处理器、存储容量扩展至 pM 时，引入函数 $G(p)$ 表示存储容量增加所带来的**问题规模放大因子**，则扩展后的工作负载为

$$W' = fW + (1 - f)G(p)W.$$

使用 p 个处理器时，Sun 和 Ni 给出的加速比为

$$\begin{aligned} S(p) &= \frac{fW + (1 - f)G(p)W}{fW + (1 - f)G(p)W/p} \\ &= \frac{f + (1 - f)G(p)}{f + (1 - f)G(p)/p}. \end{aligned}$$

该定律具有如下重要特性：

- 当 $G(p) = 1$ 时，退化为 **Amdahl 定律**。
- 当 $G(p) = p$ 时，退化为 **Gustafson 定律**。
- 当 $G(p) > p$ 时，加速效果甚至优于 Gustafson 定律，适用于计算复杂度随存储需求快速增长的问题。

若考虑并行执行的额外开销 $W_{\text{extra}}(p)$, 加速比可修正为

$$\begin{aligned} S(p) &= \frac{fW + (1-f)G(p)W}{fW + (1-f)G(p)W/p + W_{\text{extra}}(p)} \\ &= \frac{f + (1-f)G(p)}{f + (1-f)G(p)/p + W_{\text{extra}}(p)/W}. \end{aligned}$$

Sun-Ni 定律表明:

并行加速不仅受串行比例约束, 还与存储扩展能力和问题规模增长方式密切相关.

真是一场酣畅淋漓的赤史啊 🍷

The End